IOT · LAB2

# Làm việc với LED — Bài tập

*Phiếu bài tập & Mẫu nộp bài của sinh viên — FreeRTOS trên YoloUNO (ESP32-S3)*

**Môn học:** Hệ thống nhúng — Từ Arduino Uno đến ESP32-S3 và Raspberry Pi kết hợp AI · **Nền tảng:** YoloUNO (ESP32-S3)

## Thông tin sinh viên

| | |
|---|---|
| **Họ và tên** | Phạm Công Võ |
| **MSSV** | 2313946 |
| **Lớp / Nhóm** | L01 |
| **Ngày** | 05/11/2026 |

## Cách hoàn thành & nộp tài liệu này

Tài liệu này là mẫu nộp bài cho LAB 2. Hãy đọc các bước dưới đây trước khi bắt đầu.

1. Điền thông tin sinh viên vào bảng phía trên (họ tên, MSSV, lớp và ngày).
2. Giải từng bài tập trên phần cứng thật trước. Build và nạp (flash) mã lên YoloUNO bằng VS Code + PlatformIO và xác nhận mạch hoạt động đúng yêu cầu.
3. Với mỗi bài tập, dán **mã nguồn hoàn chỉnh, chạy được** của bạn (toàn bộ `main.cpp`, gồm các dòng `#include`, các hàm tác vụ và `setup()`) vào khung được cung cấp ngay bên dưới bài tập đó.
4. Giữ mã dễ đọc: giữ nguyên thụt lề, và tạo mọi tác vụ bên trong `setup()` bằng `xTaskCreate()`, tuân theo cách đặt tên trong từng bài tập (ví dụ `task_led_blinky`, `task_neo_blinky`).
5. Không xóa hoặc thay đổi kích thước các khung trả lời — khung sẽ tự mở rộng khi bạn dán thêm dòng.
6. Khi đã hoàn thành cả năm bài tập, lưu file này (giữ định dạng `.docx`, hoặc xuất ra `PDF`) và nộp **toàn bộ tài liệu**.

> **Lưu ý — chân trên YoloUNO**
> LED duy nhất trên board **nối với GPIO 48** và **NeoPixel nối với GPIO 45**. Mỗi bài tập được xây dựng dựa trên project khởi đầu trong hướng dẫn LAB 2, vì vậy bạn có thể sao chép project đó và sửa main.cpp cho từng nhiệm vụ.

> **Ghi chú chấm điểm**
> Bạn được chấm điểm dựa trên hoạt động đúng trên phần cứng và mã nguồn gọn gàng, đầy đủ được dán vào đúng khung. Khung để trống, hoặc mã không khớp với yêu cầu của bài tập, sẽ không được tính điểm.

## BÀI TẬP 1 Nhấp nháy luân phiên đơn giản

**Mục tiêu:** Cấu trúc tác vụ cơ bản và định thời.

### Yêu cầu

- Hiện thực `task_led_blinky` sao cho LED **SÁNG trong 1 giây** và **TẮT trong 2 giây**.
- Chỉ chạy **duy nhất tác vụ này** và quan sát LED nhấp nháy.

**Lời giải của bạn — mã nguồn hoàn chỉnh:**

```
#include <Arduino.h>
#define LED_PIN GPIO_NUM_48 // LED on-board YoloUNO (GPIO 48)

// Tác vụ 1: Nhấp nháy LED đơn (SÁNG 1 giây, TẮT 2 giây)
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(1000); // Sáng 1 giây

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(2000); // Tắt 2 giây
  }
}

void setup() {
  Serial.begin(115200);
  xTaskCreate(task_led_blinky,   // Hàm tác vụ
              "Task_LED_Blinky", // Tên tác vụ
              2048,              // Stack depth
              NULL,              // Tham số
              1,                 // Độ ưu tiên
              NULL               // Task handle
  );
}

void loop() {}
```

## BÀI TẬP 2 Phối hợp LED và NeoPixel

**Mục tiêu:** Làm quen với đồng bộ giữa hai tác vụ.

### Yêu cầu

- Hiện thực `task_led_blinky` với mẫu **SÁNG 2 s, TẮT 2 s**.
- Hiện thực `task_neo_blinky` để bật NeoPixel màu **ĐỎ** khi LED SÁNG, và **TẮT (ĐEN)** khi LED TẮT.
- Bạn sẽ cần một **biến dùng chung** để phối hợp hai tác vụ.

**Lời giải của bạn — mã nguồn hoàn chỉnh:**

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48 // LED on-board YoloUNO (GPIO 48)
#define NEO_PIN 45          // NeoPixel data pin (GPIO 45)
#define LED_COUNT 1         // 1 LED NeoPixel

// Biến chia sẻ trạng thái đồng bộ giữa 2 tác vụ
volatile bool led_state = false;

// Tác vụ 1: Điều khiển LED đơn (SÁNG 2s, TẮT 2s) và cập nhật biến led_state
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    digitalWrite(LED_PIN, HIGH);
    led_state = true; // Báo trạng thái LED đang SÁNG
    vTaskDelay(2000);

    digitalWrite(LED_PIN, LOW);
    led_state = false; // Báo trạng thái LED đang TẮT
    vTaskDelay(2000);
  }
}

// Tác vụ 2: Điều khiển NeoPixel theo biến led_state
void task_neo_blinky(void *pvParameters) {
  Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();
  strip.show();

  while (1) {
    if (led_state) {
      strip.setPixelColor(0, strip.Color(255, 0, 0)); // Bật ĐỎ khi LED SÁNG
    } else {
      strip.setPixelColor(0, strip.Color(0, 0, 0));   // TẮT khi LED TẮT
    }
    strip.show();
    vTaskDelay(50); // Kiểm tra lại sau 50ms
  }
}

void setup() {
```

```
  Serial.begin(115200);

  xTaskCreate(task_led_blinky,    // Hàm tác vụ
              "Task_LED_Blinky", // Tên tác vụ
              2048,              // Stack depth
              NULL,              // Tham số
              1,                 // Độ ưu tiên
              NULL               // Task handle
  );
  xTaskCreate(task_neo_blinky,
              "Task_NEO_Blinky",
              2048,
              NULL,
              1,
              NULL
  );
}

void loop() {}
```

## BÀI TẬP 3 Các mẫu nhấp nháy độc lập

**Mục tiêu:** Thực hành các chu kỳ định thời bất đồng bộ, độc lập.

### Yêu cầu

- Hiện thực `task_led_blinky` với chu kỳ **SÁNG 1,5 s / TẮT 0,5 s**.
- Hiện thực `task_neo_blinky` để NeoPixel lần lượt chuyển qua **ĐỎ (1 s), XANH LÁ (2 s) và XANH DƯƠNG (2 s)**, luôn luôn đổi màu.
- Cả hai tác vụ phải chạy **độc lập**, bất kể trạng thái của tác vụ còn lại.

**Lời giải của bạn — mã nguồn hoàn chỉnh:**

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48)
#define NEO_PIN 45
#define LED_COUNT 1

// Tác vụ 1: Nhấp nháy LED độc lập (SÁNG 1.5s, TẮT 0.5s)
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(1500); // Sáng 1.5s

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(500);  // Tắt 0.5s
  }
}

// Tác vụ 2: NeoPixel chuyển màu độc lập
void task_neo_blinky(void *pvParameters) {
  Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();
  strip.show();

  while (1) {
    strip.setPixelColor(0, strip.Color(255, 0, 0)); // ĐỎ 1 giây
    strip.show();
    vTaskDelay(1000);

    strip.setPixelColor(0, strip.Color(0, 255, 0)); // XANH LÁ 2 giây
    strip.show();
    vTaskDelay(2000);

    strip.setPixelColor(0, strip.Color(0, 0, 255)); // XANH DƯƠNG 2 giây
    strip.show();
    vTaskDelay(2000);
  }
}

void setup() {
```

```
  Serial.begin(115200);

  xTaskCreate(task_led_blinky,
              "Task_LED_Blinky",
              2048,
              NULL,
              1,
              NULL
  );
  xTaskCreate(task_neo_blinky,
              "Task_NEO_Blinky",
              2048,
              NULL,
              1,
              NULL
  );
}

void loop() {}
```

## BÀI TẬP 4 Chuỗi màu NeoPixel nâng cao

**Mục tiêu:** Hành vi phức tạp hơn và tham số hóa tác vụ.

### Yêu cầu

- Giữ `task_led_blinky` nhấp nháy theo mẫu **SÁNG 2 s, TẮT 2 s**.
- Tạo một tác vụ mới `task_neo_rainbow` cho NeoPixel lần lượt chuyển qua **năm màu khác nhau** (ví dụ ĐỎ, VÀNG, XANH LÁ, XANH NGỌC, XANH DƯƠNG), mỗi màu trong **3 s**.
- Tất cả các tác vụ phải chạy **đồng thời**; LED và NeoPixel ***không*** được đồng bộ với nhau.

**Lời giải của bạn — mã nguồn hoàn chỉnh:**

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48
#define NEO_PIN 45
#define LED_COUNT 1

// Cấu trúc RGB Color
struct RGBColor {
  uint8_t r, g, b;
};

// Mảng 5 màu cầu vồng
const RGBColor rainbowColors[5] = {
    {255, 0, 0},   // ĐỎ
    {255, 255, 0}, // VÀNG
    {0, 255, 0},   // XANH LÁ
    {0, 255, 255}, // XANH NGỌC (Cyan)
    {0, 0, 255}    // XANH DƯƠNG
};

// Tác vụ 1: LED đơn nhấp nháy SÁNG 2s, TẮT 2s
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);
  while (1) {

    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(2000);

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(2000);
  }
}

// Tác vụ 2: NeoPixel chuyển 5 màu cầu vồng, mỗi màu giữ 3 giây (3000 ms)
void task_neo_rainbow(void *pvParameters) {
  Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();
  strip.show();

  int colorIndex = 0;
```

```
  while (1) {
    RGBColor c = rainbowColors[colorIndex];
    strip.setPixelColor(0, strip.Color(c.r, c.g, c.b));
    strip.show();

    vTaskDelay(3000); // Giữ 3 giây

    // Chuyển sang màu tiếp theo
    colorIndex = (colorIndex + 1) % 5;
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(task_led_blinky,
              "Task_LED_Blinky",
              2048,
              NULL,
              1,
              NULL
  );
  xTaskCreate(task_neo_rainbow,
              "Task_NEO_Rainbow",
              2048,
              NULL,
              1,
              NULL
  );
}

void loop() {}
```

## BÀI TẬP 5 Mô phỏng hệ thống đèn giao thông

**Mục tiêu:** Mô phỏng một bộ điều khiển đèn giao thông đơn giản bằng các tác vụ bất đồng bộ.

### Các tác vụ cần hiện thực

- Tác vụ 1 (`task_led_blinky`): LED trên board phải nhấp nháy — **SÁNG 1 giây, rồi TẮT 1 giây**, liên tục.
- Tác vụ 2 (`task_traffic_light`): NeoPixel đóng vai trò đèn tín hiệu giao thông, lặp vô hạn:
  - Hiển thị **ĐỎ trong 7 giây**.
  - Sau đó hiển thị **VÀNG (AMBER) trong 2 giây**.
  - Sau đó hiển thị **XANH trong 5 giây**.
  - Lặp lại chu kỳ này vô hạn.

**Lời giải của bạn — mã nguồn hoàn chỉnh:**

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>

#define LED_PIN GPIO_NUM_48
#define NEO_PIN 45
#define LED_COUNT 1

// Cấu trúc pha đèn giao thông (Màu sắc và thời gian sáng)
struct TrafficPhase {
  uint8_t r, g, b;
  uint32_t duration_ms;
};

// Chuỗi trạng thái đèn giao thông: ĐỎ 7s -> VÀNG 2s -> XANH 5s
const TrafficPhase trafficPhases[3] = {
    {255, 0, 0, 7000},   // Đèn ĐỎ: 7 giây
    {255, 180, 0, 2000}, // Đèn VÀNG (Amber): 2 giây
    {0, 255, 0, 5000}    // Đèn XANH: 5 giây
};

// Tác vụ 1: Nhịp tim hệ thống (SÁNG 1 giây / TẮT 1 giây)
void task_led_blinky(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(1000);

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(1000);
  }
}

// Tác vụ 2: Đèn giao thông tuần tự theo chu kỳ
void task_traffic_light(void *pvParameters) {
  Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);
  strip.begin();
  strip.clear();
  strip.show();
```

```
  int phase = 0;

  while (1) {
    TrafficPhase p = trafficPhases[phase];
    strip.setPixelColor(0, strip.Color(p.r, p.g, p.b));
    strip.show();

    vTaskDelay(p.duration_ms); // Giữ màu theo thời gian quy định của từng pha

    // Chuyển sang pha tiếp theo trong chu kỳ (Đỏ -> Vàng -> Xanh -> Đỏ...)
    phase = (phase + 1) % 3;
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(task_led_blinky,
              "Task_LED_Heartbeat",
              2048,
              NULL,
              1,
              NULL
  );
  xTaskCreate(task_traffic_light,
              "Task_Traffic_Light",
              2048,
              NULL,
              1,
              NULL
  );
}

void loop() {}
```